Instead of using aliases to wrap additional functionality around greet, you can just redefine greet in a separate module and include that module instead:

part2/greet_with_super.rb
```
module EnthusiasticGreetings
  include Greetings

  def greet
    "Hey, #{super}!"
  end
end

class MyClass
  include EnthusiasticGreetings
end

MyClass.ancestors[0..2]  # => [MyClass, EnthusiasticGreetings, Greetings]
MyClass.new.greet        # => "Hey, hello!"
```

The chain of ancestors of MyClass includes EnthusiasticGreetings and then Greetings, in that order. That's why by calling greet, you end up calling EnthusiasticGreetings#greet, and EnthusiasticGreetings#greet can in turn call into Greetings#greet with super. This solution is not as glamorous as alias_method_chain, but it's simpler and all the better for it. Recent versions of ActiveRecord::Validations acknowledge that simplicity by using a regular override instead of alias_method_chain:

gems/activerecord-4.1.0/lib/active_record/validations.rb
```
module ActiveRecord
  module Validations
    # The validation process on save can be skipped by passing
    # <tt>validate: false</tt>.
    # The regular Base#save method is replaced with this when the
    # validations module is mixed in, which it is by default.
    def save(options={})
      perform_validations(options) ? super : false
    end

    # Attempts to save the record just like Base#save but will raise
    # a +RecordInvalid+ exception instead of returning +false+ if
    # the record is not valid.
    def save!(options={})
      perform_validations(options) ? super : raise(RecordInvalid.new(self))
    end

    def perform_validations(options={})
      # ...
```

Validation#save performs the actual validation (by calling the private method perform_validations). If the validation succeeds, then it proceeds with the normal